

Aula 3 — Testes de SPA & PWA

"Como testar PWA offline? Como verificar hydration?"

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 08/10/2026

Recap da Aula 2 — o que de fato fizemos

- Playwright por dentro: CDP, locators, auto-waiting, flakiness
- `await expect(...)` — `expect` de locator/page é assíncrono (e precisa de matcher)
- Network mocking com `page.route()` — spec 02, testes 1–4
- `storageState` / `auth.setup.ts` : login uma vez, reaproveitado nos specs

Ficou pra hoje: visual regression (spec 03) e os blocos 5–7 do spec 02 (opcionais).

| Dúvida no spec 02? Chama agora. O bot ainda aceita PR até o prazo no Canvas.

Agenda da aula (3h30)

- **90min** — SPA & PWA: SPA vs hydration, Service Worker, manifest, offline (teoria + demo guiada)
- **15min** — Intervalo
- **90min** — Lab hands-on: specs 03 (visual), 04 (SPA), 05 (PWA/offline) + Lighthouse CI
- **15min** — Wrap-up e entrega

Objetivos da aula

- **Testar** SPAs: esperar o app ficar pronto (hydration), navegação sem recarregar, lazy loading
- **Testar** Service Worker lifecycle e cache strategies
- **Validar** PWAs (manifest, install prompt, offline mode)
- **Configurar** Lighthouse CI bloqueando merge
- **Aplicar** performance budgets como teste

SPA × site tradicional — o que muda pro teste

Site tradicional

cada clique → o **servidor** manda uma **página nova** e a tela recarrega inteira.

SPA (Single Page App)

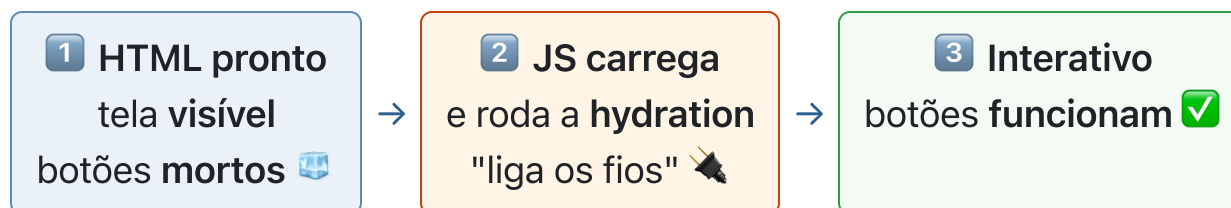
o navegador baixa o **JavaScript uma vez**; os cliques **trocam a tela sem recarregar**.

Consequências pra quem testa SPA:

1. O app **demora um instante pra ficar pronto** depois de abrir → não dá pra clicar de cara
2. Navegar **não recarrega** → o estado do JS continua vivo (o spec 04 prova isso)
3. Telas viram **chunks carregados sob demanda** (*lazy loading*)
4. Abrir uma URL profunda direto (`/movie/603`) também precisa funcionar (*deep link*)

Hydration — o que é e por que quebra teste

Em apps com **SSR** (o servidor já manda o HTML pronto, ex.: Next.js) a página aparece **antes** de o JavaScript chegar:



Hydration = o JS "acordar" o HTML que já está na tela (anexar os cliques). Se o teste clica fazendo **1**, o clique é **perdido** — e o teste falha às vezes.

O **CineFav** é uma SPA **sem SSR** (React + Vite): o HTML vem vazio e o React monta tudo no navegador. Não tem hydration, mas tem o **mesmo risco**: clicar antes de o app estar **pronto**.

O fluxo do teste — por que quebra (e por que só às vezes)

❌ Sem esperar o app ficar pronto

- 1 `page.goto('/')`
- 2 HTML na tela — **botões ainda mortos** 🛑
- 3 `click()` ← o clique cai no vazio 🖱️
- 4 `expect(...).toBeVisible()` → **falha**

✅ Esperando o sinal de pronto

- 1 `page.goto('/')`
- 2 espera `data-app-ready="true"` ⌚
- 3 `click()` ← agora o botão responde ✅
- 4 `expect(...).toBeVisible()` → **passa**

Por que falha às vezes: é uma **corrida** entre o teste e o JavaScript.

Máquina	Quem chega primeiro	Resultado
Seu notebook (rápido)	o JS liga antes do clique	✅ passa
CI / máquina lenta	o clique chega antes do JS	❌ falha

É o clássico "**passa na minha máquina e falha no CI**" (teste *flaky*). A cura não é `waitForTimeout(2000)` : é **esperar um sinal** de que o app está pronto.

Testando: esperar o app ficar pronto

```
// CineFav: o app marca <html data-app-ready="true"> quando o React terminou de montar
await page.goto('/qa');
await expect(page.locator('html')).toHaveAttribute('data-app-ready', 'true');

await page.getByTestId('movielist-search-button').click(); // agora o clique é seguro
await expect(page.getByTestId('search-screen')).toBeVisible();
```

- **Esperar um sinal de pronto** (atributo, elemento, texto), não um tempo fixo (`waitForTimeout`)
- Em apps **SSR** (Next.js) o princípio é o mesmo: espere algo que **só existe depois de hidratar**
- Evite `networkidle` como regra: é um palpite ("a rede sossegou"), não prova que o app está interativo
- Cada ação do Playwright (`click` , `fill`) já **espera o elemento ficar clicável** — mas não sabe se o JS ligou

É o **spec 04** do lab (teste 1): `data-app-ready` .

O que é um Service Worker (SW)?

Um **Service Worker** é um **script** que o navegador roda em **segundo plano**, separado da página. Pense num **porteiro** entre o seu app e a internet: toda requisição passa por ele.

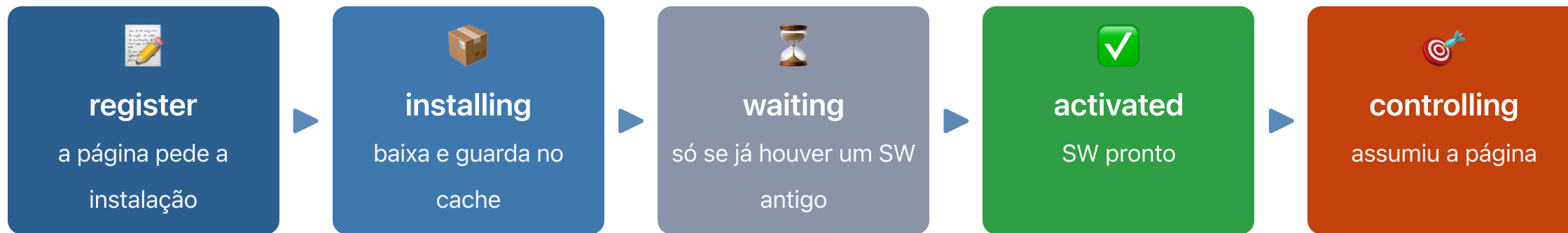


- **Intercepta** cada `fetch` do app e escolhe: responder do **cache** ou ir na **rede**
- É por isso que a PWA **abre sem internet**: o porteiro entrega da prateleira (cache)
- Não tem acesso à tela (não mexe no DOM) · só existe em **HTTPS** (ou `localhost`)

Sem SW não há offline — e é isso que o Playwright precisa conferir.

Ciclo de vida do SW — em uma imagem

Primeira visita (o SW nasce e instala):



⚠ **janela de perigo** — ativo, mas **ainda não controla** a página

✍ Onde o teste espera

1. `active.state === 'activated'`
2. `controller !== null` ← só então derrubar a rede

🔄 Versão nova (atualização)

installing → **waiting** (espera o usuário clicar "**Atualizar**") →
activated → `controllerchange`

Teste offline que **pula a etapa** 🎯 cai exatamente na janela de perigo — e falha às vezes.

Ciclo de vida do SW — e por que isso complica o teste

Fase	O que acontece	No teste
register	a página pede ao navegador pra instalar o SW	só no build (<code>npm run preview</code>) — não no <code>npm run dev</code>
installing	baixa os arquivos e guarda no cache	esperar terminar
waiting	versão nova espera a antiga fechar	é o toast "Nova versão — Atualizar"
activated	SW pronto	<code>reg.active.state === 'activated'</code>
controlling	o SW assumiu a página	<code>navigator.serviceWorker.controller !== null</code>

⚠ **activated** ≠ **controlando**. Na 1ª visita o SW ativa, mas só passa a **controlar** a página depois. Teste offline sem esperar o `controller` falha de forma intermitente.

Testando o SW com Playwright — o essencial

```
test('SW ativo e controlando a página', async ({ page }) => {  
  await page.goto('/qa'); // 1. visita online → o SW registra e instala  
  
  await expect  
    .poll(() => // 2. espera o SW ficar "activated"  
      page.evaluate(async () => {  
        const reg = await navigator.serviceWorker.getRegistration();  
        return reg?.active?.state;  
      })),  
    .toBe('activated');  
  
  await page.waitForFunction( // 3. ...e CONTROLAR esta página  
    () => navigator.serviceWorker.controller !== null,  
  );  
});
```

Roda contra o `build ( vite preview ).O playwright.config.ts` do lab já faz isso.

Estratégias de cache — qual usar e como testar

Estratégia	Ideia	No CineFav	Como testar
CacheFirst	cache primeiro; rede só se faltar	posters do TMDb (não mudam)	<code>caches.open('cinefav-posters')</code> tem entradas
NetworkFirst	rede primeiro; cache se cair	<code>/api/movies.json</code> (dados mudam)	<code>setOffline(true) + reload</code> → ainda lista os filmes
StaleWhileRevalidate	mostra o cache já e atualiza ao fundo	(não usado)	conferir o dado velho na hora e o novo depois

O app shell (HTML, JS, CSS) vai no *precache*: o SW guarda tudo na instalação. É o que faz o app abrir offline.

Atualização do SW — o toast "Nova versão"

Quando você publica uma versão nova, o SW novo **espera (waiting)** até a antiga fechar. A PWA do CineFav **avisa o usuário**, e ele escolhe quando atualizar:

1. SW novo instala e fica **waiting**
2. app mostra o toast " ✨ **Nova versão — Atualizar**"
3. clique → o SW novo assume (`controllerchange`) e a página recarrega

Testar isso de ponta a ponta pede **dois builds** (versão A e B). No lab conferimos o essencial: o SW fica **ativo e controlando**. Os cenários de *waiting* são onde mora o bug.

PWA — manifest validation

```
test('manifest is valid', async ({ page }) => {  
  const response = await page.goto('/manifest.json');  
  const manifest = await response?.json();  
  
  expect(manifest.name).toBeTruthy();  
  expect(manifest.short_name).toBeTruthy();  
  expect(manifest.start_url).toBe('/');  
  expect(manifest.display).toBe('standalone');  
  expect(manifest.icons.length).toBeGreaterThan(0);  
  expect(manifest.icons.find((i) => i.sizes === '192x192')).toBeDefined();  
  expect(manifest.icons.find((i) => i.sizes === '512x512')).toBeDefined();  
});
```

PWA — install prompt simulation

```
test('install prompt fires on supported browsers', async ({ page }) => {
  let installPromptFired = false;

  await page.exposeFunction('onBeforeInstallPrompt', () => {
    installPromptFired = true;
  });

  await page.addInitScript(() => {
    window.addEventListener('beforeinstallprompt', () => {
      (window as any).onBeforeInstallPrompt();
    });
  });

  await page.goto('/');
  // Trigger criteria do browser

  expect(installPromptFired).toBe(true);
});
```

PWA — offline mode test

```
test('catálogo continua acessível offline', async ({ page, context }) => {  
  await page.goto('/qa');  
  await expect(page.getByTestId('movielist-grid')).toBeVisible();  
  
  // SW "activated" NÃO basta: precisa CONTROLAR a página  
  await page.waitForFunction(() => navigator.serviceWorker.controller !== null);  
  
  await context.setOffline(true); // perda total de rede  
  await page.reload();  
  
  await expect(page.getByTestId('movielist-grid')).toBeVisible();  
  await expect(page.getByTestId('offline-banner')).toBeVisible();  
});
```

Sequência: visita online → SW controla → offline → reload. Pular o 2º passo é o erro mais comum.

Site comum x PWA — o que muda de verdade

	Site comum	PWA
Sem internet	✗ dinossauro do Chrome	✓ Service Worker serve do cache
Na tela inicial	✗ só favorito	✓ instalável (manifest)
Janela	aba + barra de endereço	standalone (parece app)
Atualização	a cada visita	controlada (toast "nova versão")
Dados locais	somem fácil	IndexedDB + storage persistente
Recursos do aparelho	✗	atalhos, Badging , Web Share

No CineFav: abra `/pwa` ("Raio-X") — lê **ao vivo** HTTPS, manifest, SW, cache, banco e rede. Cada linha tem `data-testid` → vira asserção.

Raio-X da PWA — o app se auto-diagnostica

← Voltar

Raio-X da PWA

Este app é uma **Progressive Web App**: o mesmo site, mas instalável, com cache offline, banco local e recursos do dispositivo. Tudo abaixo é lido ao vivo do navegador.

Contexto seguro (HTTPS)

http://localhost:4173

Manifest do app

nome, ícones e cores válidos

Service Worker

estado: activated

SW controlando esta página

sim — intercepta as requisições

Cache Storage (arquivos offline)

2 cache(s) · 36 arquivo(s)

Banco local (IndexedDB)

1 registro(s) salvo(s)

Rede

online

Rodando como app instalado

em uma aba do navegador

Armazenamento: 112.4 MB de 2160.4 MB · pode ser limpo pelo navegador

Recursos do dispositivo

Instalar (indisponível agora)

Compartilhar (copia o link)

Badge no ícone (0 favoritos)

Pedir armazenamento persistente

Limpar cache, banco e SW

Site comum x PWA

Recurso	Site comum	PWA
Abre sem internet	erro do navegador	Service Worker serve do cache
Ícone na tela inicial	só favorito	instalável (manifest)
Janela própria	aba com barra de endereço	standalone
Dados locais	some ao limpar	IndexedDB + storage persistente

Roda no **build** (`npm run build && npm run preview`): em `npm run dev` não há Service Worker, então o painel mostra tudo apagado — bom pra **comparar**.

Prof. Jackson Smith Moisés Matias — 08/10/2026

19

"Banco offline": 2 lugares, 2 papéis

	Cache Storage (Service Worker)	IndexedDB (código do app)
Guarda	arquivos e respostas HTTP prontas	dados estruturados que o app lê/escreve
No CineFav	app shell, <code>movies.json</code> , posters TMDB	respostas da API TMDB, comentários
Estratégia	<code>CacheFirst</code> / <code>NetworkFirst</code> (Workbox)	rede primeiro → banco como plano B
Como testar	<code>caches.open('cinefav-posters')</code>	<code>indexedDB.open('cinefav')</code>

Fila offline (outbox): comentou sem rede → grava local 🕒 → volta a rede → ✓ sincronizado. Mesmo padrão offline-first do lab Flutter.

Features de dispositivo — e como testar cada uma

Feature	API	Teste Playwright
Instalar	<code>beforeinstallprompt</code>	<code>dispatchEvent</code> do evento falso → botão habilita
Atalhos do ícone	<code>manifest.shortcuts</code>	<code>fetch(manifest)</code> e conferir as URLs
Atualização	<code>registerType: 'prompt'</code>	toast "Nova versão" + botão Atualizar
Badge no ícone	Badging API	feature-detect (só app instalado)
Compartilhar	Web Share	fallback copia o link
Offline real	<code>context.setOffline(true)</code>	reload + <code>offline-banner</code>

Bônus (não pontua): `tests/e2e-bonus/09-pwa-painel` e `10-banco-offline` — rode `npm run test:bonus`.

Background Sync — extra (fora do lab)

```
test('queues message for sync when offline', async ({ page, context }) => {
  await page.goto('/messages');
  await context.setOffline(true);

  await page.fill('[name=message]', 'Olá!');
  await page.click('button[type=submit]');

  // Mensagem fica em estado "pending sync"
  await expect(page.getByText('Pending sync')).toBeVisible();

  // Voltar online
  await context.setOffline(false);

  // Background Sync deve disparar
  await page.evaluate(async () => {
    const reg = await navigator.serviceWorker.ready;
    return reg.sync.register('flush-outbox');
  });

  await expect(page.getByText('Sent')).toBeVisible({ timeout: 5000 }); // auto-wait, sem sleep
});
```

Lighthouse CI — visão geral

```
npm install -g @lhci/cli@0.13.x
```

```
# rodar local  
lhci autorun
```

```
// lighthouserc.json  
{  
  "ci": {  
    "collect": {  
      "url": ["http://localhost:3000/"],  
      "numberOfRuns": 3  
    },  
    "assert": {  
      "assertions": {  
        "categories:performance": ["error", { "minScore": 0.9 }],  
        "categories:pwa": ["error", { "minScore": 0.9 }],  
        "categories:accessibility": ["error", { "minScore": 0.9 }],  
        "first-contentful-paint": ["warn", { "maxNumericValue": 2000 }],  
        "largest-contentful-paint": ["error", { "maxNumericValue": 2500 }]  
      }  
    }  
  }  
}
```

Performance budgets

Conceito de **Tim Kadlec (2013)** popularizado por web.dev. No lab, os budgets estão no `lighthouse.json`:

```
"assertions": {  
  "categories:performance": ["error", { "minScore": 0.9 }],  
  "categories:accessibility": ["error", { "minScore": 0.9 }],  
  "categories:best-practices": ["error", { "minScore": 0.9 }],  
  "first-contentful-paint": ["error", { "maxNumericValue": 2000 }],  
  "largest-contentful-paint": ["error", { "maxNumericValue": 2500 }],  
  "total-blocking-time": ["error", { "maxNumericValue": 300 }]  
}
```

Também dá pra limitar tamanho de bundle (`resource-summary:script:size`). **Death by a thousand cuts** evitado.

Lighthouse CI no GitHub Actions

```
name: Lighthouse CI
on: [pull_request]

jobs:
  lighthouse:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci
      - run: npm run build
      - run: npm install -g @lhci/cli@0.13.x
      - run: lhci autorun
    env:
      LHCI_GITHUB_APP_TOKEN: ${ secrets.LHCI_GITHUB_APP_TOKEN }
```

Resultado posta comment no PR com link pra report. Falha bloqueia merge.

IA aplicada à análise Lighthouse

Em vez de só ler números, **alimentar relatório pra LLM**:

```
Você é especialista em performance web. Analise este Lighthouse report:
```

```
[JSON do report]
```

```
Identifique:
```

1. Quais regressões são significativas (semanticamente)?
2. Quais são triviais (variação de medição)?
3. Quais ações concretas recomendaria?

```
Responda em formato estruturado.
```

IA faz **interpretação semântica**, não só comparação numérica.

Cobertura de código — o que o relatório mede

Cobertura = **quanto do código-fonte foi executado** pelos testes. Quatro métricas:

Métrica	Pergunta que responde
Statements	quantas instruções rodaram?
Branches	os dois lados de cada <code>if / ?:</code> rodaram?
Functions	cada função foi chamada ao menos 1 vez?
Lines	quantas linhas rodaram?

No e2e do CineFav: cobertura **V8 do Chromium** (`page.coverage.startJSCoverage()`) + sourcemap → relatório por arquivo de `src/`.

Relatório real — suíte e2e do CineFav

All files

78.47% Statements 113/14467.39% Branches 62/9258.82% Functions 60/10277.85% Lines 109/140

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
App.tsx	100%	7/7	100%	0/0
components	75%	15/20	58.33%	7/12
main.tsx	100%	1/1	100%	0/0
screens	84%	42/50	74.46%	35/47
services	72.72%	24/33	47.36%	9/19
store	71.87%	23/32	78.57%	11/14
utils	100%	1/1	100%	0/0

Code coverage generated by Istanbul at 2026-10-03T19:01:00.670Z

78% statements · 67% branches · 59% functions. No vermelho: services (branches 47%) e utils (functions 44%).

Dentro de um arquivo — o que ficou de fora

All files / store favorites.ts

57.89% Statements 11/19 62.5% Branches 5/8 55.55% Functions 5/9 55.55% Lines 10/18

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

```
1 // src/store/favorites.ts
2 //
3 // Favoritos persistidos em localStorage, com useSyncExternalStore pra
4 // qualquer tela reagir a mudanças (mesma semântica do favoritesStore mobile).
5
6 import { useSyncExternalStore } from 'react';
7
8 26x const KEY = 'cinefav-favorites';
9
10 type Listener = () => void;
11 const listeners = new Set<Listener>();
12
13 13x function read(): number[] {
14 13x   const raw = localStorage.getItem(KEY);
15 13x   if (!raw) return [];
16   try {
17     return JSON.parse(raw) as number[];
18   } catch {
19     return [];
20   }
21 }
22
23 26x let snapshot: number[] = [];
24 let snapshotRaw = '';
25
26 277x function getSnapshot(): number[] {
27 277x   const raw = localStorage.getItem(KEY) ?? '[]';
28 277x   if (raw !== snapshotRaw) {
29     snapshotRaw = raw;
30     snapshot = read();
31   }
32   return snapshot;
33 }
34
```

Verde = executado (13x , 277x). Vermelho = nunca executado: o `try/catch` do `JSON.parse` em `store/favorites.ts` .

Cobertura ≠ qualidade

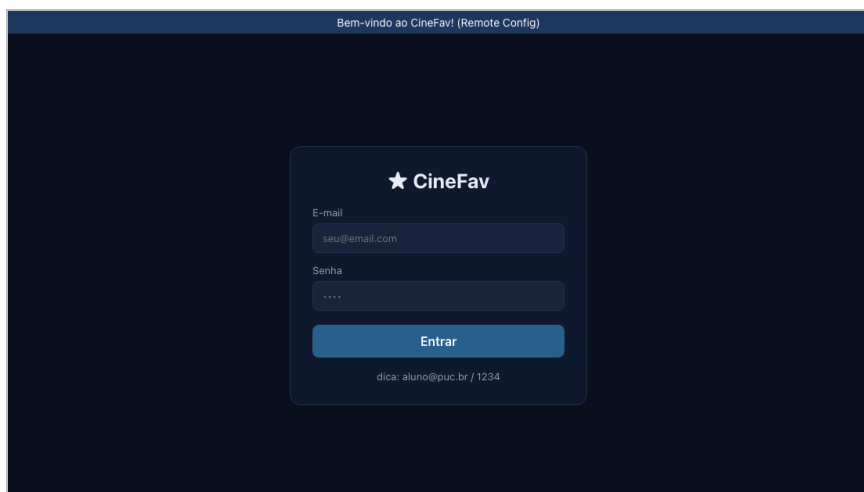
- Linha executada **não** é comportamento verificado: um teste **sem** `await` / `matcher` executa o código e dá 100% de cobertura sem checar nada
- Use o relatório pra achar o que **não** foi testado (o vermelho), não como meta de nota
- Gate sensato no CI: piso por **branches** (ex.: 70%), não 100%

Na Aula 6: por que **mutation testing** prova o que a cobertura só sugere.

Demo — Firebase Remote Config no CineFav

Mudar o app **sem novo deploy**: o banner do topo vem do parâmetro `banner_message` do Firebase.

```
cd exercicios/01-lab-web-pwa-playwright/pratica && ls .env.example  
# cole as 3 variáveis VITE_FIREBASE_* no .env (o professor passa em aula)  
npm run dev      # banner aparece no topo; sem credencial, app normal
```



Professor edita `banner_message` no console → **F5** → texto novo. Sem credencial: nenhuma chamada extra.

Testando API de terceiro — sem bater no Firebase

Em teste, **não** chamamos o Firebase de verdade (rede + credencial = flaky). Interceptamos:

```
await page.route('**/firebaseremoteconfig.googleapis.com/**', (route) =>
  route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ entries: { banner_message: 'Promo de teste ativa!' } }),
  }),
);
await page.goto('/qa');
await expect(page.getByTestId('remote-banner')).toHaveText('Promo de teste ativa!');
```

Bônus (não pontua): `tests/e2e-bonus/08-remote-config-banner.spec.ts` — `npm run test:bonus`. Mesmo `page.route()` da Aula 2.

Intervalo

15 minutos

*Na volta: vamos pro **lab** — specs 03 (visual), 04 (SPA), 05 (PWA/offline) + Lighthouse CI.*

Aproveite pra abrir o terminal em `exercicios/01-lab-web-pwa-playwright/pratica` e rodar `npm install` e `npx playwright install chromium`.

Lab — specs 03, 04 e 05 + Lighthouse (90min)

```
cd exercicios/01-lab-web-pwa-playwright/pratica · ls tests/e2e
```

1. **Spec 03 — visual regression** (25min): `toHaveScreenshot` , 3 viewports, máscara. Teoria nos slides da Aula 2
2. **Spec 04 — SPA** (20min): `data-app-ready` , navegação client-side, lazy chunk
3. **Spec 05 — PWA/offline** (30min): SW ativo, manifest, offline com controller
4. **Lighthouse CI** (15min): `lighthouseci.json` + 3 budgets rodando

Tudo em `/qa` (ambiente determinístico), contra o **build** (`vite preview`) — SW não existe no dev server.

Pitfalls comuns

- ✗ Testar SPA sem aguardar hydration — falha intermitente
- ✗ Reset de cache Service Worker entre tests — esquecer = test pollution
- ✗ Lighthouse CI rodando uma vez só — variação alta, configure 3+ runs
- ✗ Budget muito apertado — bloqueia PRs legítimos por 10ms
- ✗ Não monitorar Web Vitals em produção — divergência entre lab e prod
- ✗ Offline sem esperar o SW controlar a página — `activated` não basta, falta `controller`
- ✗ `expect` de locator sem `await` — verde sem checar, ou erro "page closed"

Atividade — Lab Web + PWA (20 pts, fecha nesta aula)

Entrega: fork + PR. Prazo: ver Canvas. Enunciado completo no repo.

#	Critério	Pontos
1	Auth state reuse (<code>storageState</code>)	3
2	Network mocking (<code>route()</code>)	3
3	Visual regression, 3 viewports	4
4	SW lifecycle testado	3
5	Offline mode (<code>setOffline</code>)	3
6	Lighthouse CI, 3 budgets	4

Eliminatório: suíte 100% verde em 3 runs consecutivos.

Leituras obrigatórias

- Archibald, J. — *Service Workers Testing Strategies* (web.dev)
- web.dev — *Lighthouse CI* (oficial)
- Lock, A. — *Testing Service Workers* (artigo)
- **Mobile.dev Docs** — preparação Aula 4

Próxima aula — Aula 4

Automação Mobile com Maestro

- YAML declarativo
- Conditional flows + runFlow
- Maestro Studio
- Comparativo Maestro vs Appium vs Detox
- Cloud devices

Bons estudos!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith